Ders 1 Çalışma Özeti

Ders 1 Çalışma Özeti

Genel Konular

- **Veri ve Veritabanı Temel Kavramları**
  - Veri: Anlamı olan, kaydedilen gerçekler (isim, adres, telefon vb.). Bilgisayar içinde binary olarak saklanır, kodlanmış bit dizileri anlamlı bilgiler oluşturur.
  - Veri (ANSI tanımı): Olguların, kavramların veya talimatların insan tarafından veya otomatik yolla iletişime uygun biçimde ifadesi. JSON gibi formatlarla makineler arası ortak dil sağlanır.
  - Veritabanı: Birden çok uygulama tarafından kullanılan, gereksiz yinelemelerden arınmış, düzenli saklanan, birbiriyle ilişkili ve uyumlu, sürekli fakat statik olmayan, belirli bir amaç için bir araya getirilmiş veri topluluğu — "küçük bir dünya".
- **Veritabanı Tanımının 6 Temel Maddesi**
  - Birden çok uygulama tarafından kullanılması: Farklı programlama dilleri, farklı ortamlar (web, konsol, IDE) üzerinden erişim.
  - Birçok kişiye bakması: Üniversite örneğinde öğrenci, hoca, rektör, dekan gibi birçok kullanıcı profiline hizmet etmesi.
  - Gereksiz yinelemelerden arınması: Aynı verinin birden çok yerde kopyasının bulunmasının sakıncaları — güncelleme tutarsızlığı, veri doğruluğunun (data integrity) riske girmesi. Gereksiz olmayan yinelemeler (yabancı anahtar, indeks) kontrollü kabul edilir.
  - Düzenli bir şekilde saklanması: Fiziksel düzenleme — kayıt dizilerinin sıralı saklanması. Sıralama sayesinde binary search (logaritmik arama) ile 10 milyon kayıtta ~16-17 adımda erişim. Hash tabloları ile O(1) erişim mümkün, ancak aralık sorguları için B-tree (k-yollu arama ağaçları) tercih edilir.
  - Sürekli fakat statik olmaması: Disk üzerinde non-volatile (uçucu olmayan) olarak saklanır, makine kapanınca veri korunur. Ancak arşiv de değildir — SQL ile kayıt değiştirilebilir, güncellenebilir.
  - Belirli bir amaç için bir araya getirilmesi: Veri saklanmasının nihai amacı sorgulamaktır. Veri tabanına erişim dilleri (SQL) ile sorgulama yapılır.
- **Veritabanı Yönetim Sistemi (DBMS / VTYS)**
  - Veritabanı sistemiyle ilgili her türlü işletimsel gereksinimi karşılamak için kullanılan sistem seviyesinde karmaşık, merkezi yazılım sistemi.
  - İşletim sistemi kaynakları (ana hafıza, disk, işlemci) VTYS'ye tahsis edilir. VTYS bu kaynakları çok kullanıcılı ortamda adil ve verimli şekilde yönetir.
  - İşletim sistemi ve kullanıcılar veritabanına doğrudan erişemez; yalnızca VTYS üzerinden kontrollü erişim sağlanır.
- **VTYS'nin Sağladığı Olanaklar**
  - Veritabanının tanımlanması, gerçeklenmesi, kullanımı ve paylaşımı.
  - Kontrollü veri tekrarı: İndeksler otomatik olarak oluşturulur (özellikle primary key üzerine). Sistem en çok erişilen alanları izleyerek otomatik indeks yönetimi yapabilir (automatic system administration).
  - Verimli erişim metodları: İndeks yapıları — ağaç organizasyonları (B-tree, sıralama esaslı, k-yollu) ve hash organizasyonları (eşitlik esaslı).
  - Çok kullanıcılı hizmet: Farklı kullanıcı profillerine farklı arayüzler (web, konsol, grafiksel).
  - Eşzamanlılık (concurrency): Birden çok kullanıcının aynı anda güvenli erişimi.
  - Veri kurtarma ve yedekleme: Transaction (hareket) kavramı — "ya hep ya hiç" (all-or-

nothing). Uçak + otel rezervasyonu örneği: İkisi bir bütün olarak çalışmalı, elektrik kesilirse veya disk arızası olursa log dosyalarından geri sarılarak tutarlılık sağlanır.
  - İş kısıtlamaları (constraints): Ders ön koşul kontrolü, not ortalaması kontrolü gibi kurallar — stored procedure ve trigger olarak sisteme yüklenir.
  - Güvenlik: Kullanıcı bazlı erişim yetkilendirme — okuma ve yazma ayrı ayrı kısıtlanabilir. Sınav sorularının belli saatten önce görülememesi örneği.
- **Veritabanı Sistem Dosyaları (4 Temel Dosya Tipi)**
  - **Veri Dosyaları**: Veri kayıtlarının saklandığı büyük binary (random access) dosyalar. Öğrenci kayıtları, bölüm kayıtları gibi.
  - **İndeks Dosyaları**: Veriye hızlı erişim için kullanılan yardımcı dosyalar. Primary key üzerine otomatik oluşturulur. Kullanıcı da ek indeks oluşturabilir. İndeks olmadan büyük dosyalarda sırayla tarama çok zaman alır.
  - **Log Dosyaları**: Tüm işlemlerin (ekleme, çıkarma, update, okuma) kaydı. Append-only, sıralı tarama yapılır. Transaction tutarlılığı ve veri kurtarma için kritik. Sistem arızasından sonra log'dan geri sarılarak tutarlı noktaya ulaşılır.
  - **Veri Sözlüğü (Data Dictionary / Metadata)**: Üst veri — veritabanının yapısına dair bilgiler. Tablo isimleri, nitelikler, veri tipleri (varchar(10), double), tablo büyüklüğü, disk sayfası sayısı, niteliklerin değer dağılımı, erişim frekansları. Sorgu optimizasyonu (query optimization) bu bilgilerle yapılır. İndekslerin sisteme girip çıkması takip edilir. Fine-tuning (ince ayar) için kritik — günümüzde makine öğrenmesi algoritmalarıyla otomatik fine-tuning araştırmaları yapılmaktadır.
- **Veri Modeli ve Veri Modelleme**
  - Küçük dünyanın (üniversite, market, bakanlık vb.) makineye taşınması için modellenmesi gerekir.
  - ER diyagramları ve UML diyagramları ile modelleme yapılır.
  - İlişkisel model (relational model): Katı kuralları olan, tablo tabanlı model. Tablolar, nitelikler, yabancı anahtarlar ile bağlantılar.
  - NoSQL: "Not Only SQL" — esnek veri karakteristikli ortamlar için (IoT, sosyal medya, büyük veri). Şema yok, key-value, doküman, graf tabanlı modeller. İlişkisel modelin katılığını esnetmek için ortaya çıkmıştır. SQL'i tamamen değiştirmemiştir.
  - Veri ambarları (data warehouses): OLAP sistemleri, çok boyutlu küplerde saklanan önceden hesaplanmış bilgiler.
- **Transaction (Hareket / İşlem)**
  - Birden çok veritabanı eylemi gerektiren program parçacıkları.
  - ACID kriterleri: Atomiklik (ya hep ya hiç), Consistency (tutarlılık), Isolation (yalıtım), Durability (kalıcılık).
  - Sistem bu kriterleri otomatik olarak sağlar — log dosyaları ve recovery mekanizmaları ile.
- **Uygulama Programları ve Sorgular**
  - Uygulama programı: Veritabanına sorgu göndererek veri erişimi yapan program (stored procedure veya üst seviye uygulama).
  - Sorgu (query): Veritabanından verinin retrieve edilmesi (erişilmesi). Sorgu ile birlikte veri manipülasyonu da yapılabilir (ekleme, çıkarma, güncelleme).
- **Dersin Kapsamı ve Kaynaklar**
  - Dersin amacı: VTYS temel kavramlarını anlamak, ilişkisel modeli ve SQL'i öğrenmek/uygulamak, veritabanı tasarlamak.
  - Türkçe kaynak: Münali Yarımağan — "Veritabanı Sistemleri".
  - İngilizce kaynaklar: Cornel Sinavati, Ebru Skorin, Cornell Üniversitesi kitabı (Cow Book — üzerinde inek resmi olan).
  - Ders akışı: Giriş → Veri modelleme (ER/UML) → İlişkisel model → SQL → Laboratuvar uygulamaları.

Hocanın Özellikle Vurguladığı Kısımlar

- **Veri tekrarının iki ucu var**: Ne tek bir yerde olmalı ne de birçok yerde. Gereksiz yineleme olmamalı ama kontrollü yineleme (yabancı anahtar, indeks) sistemin yürümesi için gerekli. Bu dengeyi kurmak veritabanı tasarımının esaslarından.
  - Veri tekrarının avantajı: Yedekleme, daha hızlı erişim (join gereksinimi azalır).
  - Dezavantajı: Güncelleme zorluğu — birden çok kopya tutarsız kalır, veri doğruluğu

(data integrity) bozulur, sistem hantallaşır.

- **Sıralama ve erişim hızı farkı**: 10 milyon kayda sırayla bakmak vs. binary search ile 16-17 adımda erişmek arasındaki logaritmik iyileşme çok kıymetli. Hash tablosu ile O(1) erişim daha da etkileyici ama aralık sorguları için B-tree tercih ediliyor.
- **Veritabanı dosyaları binary'dir**: ASCII/Word dosyası gibi açıp okunamaz. Random access file olarak byte dizisi şeklinde saklanır. Formatını yalnızca VTYS bilir. Bu hem güvenlik hem verimlilik sağlar.
- **Transaction bütünlüğü**: Uçak + otel rezervasyonu örneği — ya hep ya hiç çalışmalı. Elektrik kesilirse, disk arızası olursa log dosyalarından geri sarılarak tutarlı noktaya ulaşılır. Bu, sistemin karmaşıklığını artıran temel unsurlardan.
- **Metadata (üst veri) sorgu optimizasyonu için kritik**: Tablo büyüklüğü, disk sayfası sayısı, niteliklerin değer dağılımı ve erişim frekansları — yanlış indeks seçimi sorgu hızını binlerce kat etkileyebilir.
- **VTYS bağımsız bir sistemdir**: İşletim sistemi bile veritabanına doğrudan erişemez. Kaynaklar VTYS'ye tahsis edilir, VTYS kendi içinde bu kaynakları yönetir. VTYS'lerin sıralaması (Oracle, SQL Server, MongoDB) kaynakları ne kadar iyi kullandıklarına bağlıdır.

Kısa Tekrar Notları

- Veri = anlamı olan, kaydedilen gerçekler (binary olarak saklanır)
- Veritabanı = düzenli, ilişkili, sürekli, sorgulanabilir veri topluluğu (küçük dünya)
- VTYS = veritabanını yöneten karmaşık sistem seviyesi yazılım
- 6 temel VTYS olanağı: tanımlama, gerçekleme, kullanım/paylaşım, kontrollü tekrar, verimli erişim, çok kullanıcılı hizmet
- 4 dosya tipi: veri dosyaları, indeks dosyaları, log dosyaları, veri sözlüğü (metadata)
- İndeks yapıları: ağaç (B-tree, sıralama esaslı) ve eş (hash) organizasyonları
- Transaction = ya hep ya hiç (ACID: Atomiklik, Consistency, Isolation, Durability)
- Log dosyaları = append-only, tüm işlemlerin kaydı, recovery için kritik
- Metadata = tablo yapısı, nitelikler, veri tipleri, değer dağılımı, erişim frekansları
- İlişkisel model = katı kurallı tablo tabanlı model; NoSQL = esnek, şemasız alternatif
- SQL = sorgulama dili; orta seviyeye gelmemiz bekleniyor
- ER diyagramları = veritabanı tasarımında küçük dünyanın modellenmesi

Detaylı Açıklamalar

- **Veritabanı neden karmaşık bir sistemdir?**: Veritabanı yönetim sistemleri, ilk bakışta basit görünen "veri sakla ve getir" işinin arkasında müthiş bir düzenleme ve optimizasyon gerektirir. Google'ın Bigtable'ında terabyte'larca, petabyte'larca veri içinde bir kelime arandığında 45 milisaniyede sonuç gelmesi; arada cache'ler, tampon bölgeler, indeks yapıları ve çok gelişmiş algoritmalar sayesindedir. Sadece dosyayı açıp sırayla kayıt okumak değildir — milyonlarca eşzamanlı kullanıcının hem okuma hem yazma yaptığı, verinin doğruluğunun (veri bütünlüğü) korunması gereken, arka planda B-tree, hash gibi veri yapılarıyla donatılmış karmaşık bir mimari söz konusudur. Bu karmaşıklığı anlamak 8-10 yıl sürebilir; bu ders ilk adımdır.
- **Veri tekrarının dengelenmesi**: Bir öğrencinin bilgilerinin üniversite veritabanında birden çok yerde (bölüm, dekanlık, yemekhane, özlük işleri) saklanması düşünüldüğünde, adres değişikliği durumunda tüm kopyaların güncellenmesi gerekir. Güncelleme sürecinde sistem tutarsız kalır — bir uygulama Beşiktaş görürken diğeri Bayrampaşa görür. Bu, veri doğruluğunu (data integrity) ciddi şekilde riske atar. Ancak hiç tekrar olmaması da iyi değildir — yabancı anahtar (foreign key) tablolar arası bağlantı için gerekli bir tekrardır, indeksler de hızlı erişim için kontrollü tekrardır. VTYS'nin görevi bu dengeyi kurmaktır.
- **Erişim metodlarının karşılaştırılması**: Bir dizide 10 milyon kayıt sırasızsa, aradığımız kaydı bulmak için en kötü ihtimalle 10 milyon adımda tarama gerekir. Kayıtlar sıralıysa binary search ile $\log_2(10.000.000) \approx 23$ adımda bulunabilir — bu çok büyük bir iyileşmedir. Hash tablosu ile başarılı bir hash fonksiyonu kullanıldığında, dosya boyutundan bağımsız

olarak doğrudan hedefe erişilir (O(1)). Ancak hash fonksiyonu sadece nokta sorgusu (equality) için uygundur; aralık sorguları (range queries) için B-tree (k-yollu arama ağaçları) çok daha avantajlıdır. Bu nedenle VTYS'lerde genel kullanım B-tree indekslerdir.

- **Transaction ve veri kurtarma mekanizması**: Gerçek hayat örneklerinde işlemler bir bütün (transaction) olarak tanımlanır. Uçak bileti + otel rezervasyonu bir bütündür — ya ikisi de olacak ya da hiçbiri. Ancak bu işlemler anında (sihirli değnek gibi) gerçekleşmez; milisaniye mertebesinde bir süreç izler. Bu süreçte elektrik kesilirse veya disk arızası olursa sistem ortada kalır. Log dosyaları tüm işlemleri (ekleme, çıkarma, güncelleme) sırayla kaydeder. Sistem tekrar ayağa kalktığında log dosyaları taranarak tutarsız durum geri sarılır (rollback) ve veritabanı tutarlı bir noktaya getirilir. Bu mekanizma olmasaydı bankacılık işlemlerinde para kaybolur, rezervasyonlar yarım kalırdı.
- **Veri sözlüğü (metadata) ve sorgu optimizasyonu**: Veri sözlüğü, veritabanının "üst verisi"dir — verinin kendisi değil, veri hakkında bilgidir. Tablo isimleri, nitelikler ve veri tipleri (varchar, double), tablo büyüklüğü (kaç megabyte, kaç disk sayfası), niteliklerin değer dağılımı (not ortalaması 0-4 arasında nerede yoğunlaşmış), erişim frekansları gibi bilgiler sorgu optimizasyonu için hayati önem taşır. Bir SQL sorgusunda FROM cümlesindeki tabloların hangi sırayla erişileceği, hangi indekslerin kullanılacağı bu istatistiklere göre belirlenir. Yanlış indeks seçimi veya gereksiz indeks kullanımı sorgu hızını binlerce kat etkileyebilir. Günümüzde bu ince ayar (fine-tuning) için makine öğrenmesi algoritmalarıyla otomatik sistem yönetimi araştırmaları yapılmaktadır.
- **NoSQL ve esnek veri ortamları**: İlişkisel model katı kuralları olan bir modeldir — tablo yapısı, nitelikler, veri tipleri bellidir. Ancak günümüzde IoT sistemleri, sosyal medya, GPS verisi gibi ortamlarda veri çok hızlı üretilmekte, sürekli güncellenmekte ve çeşitlilik göstermektedir. Bir şehirdeki milyonlarca mobil aracın 30-60 saniyede bir ürettiği GPS verisi, kaza anında belirli bir bölgeye yoğunlaşan sorgu paternleri, uzamsal veri özellikleri — bunlar ilişkisel modelin katılığını kaldırmaz. NoSQL ("Not Only SQL") bu esnek ihtiyaçlara cevap olarak ortaya çıkmıştır: şema yok, key-value, doküman, graf tabanlı modeller söz konusudur. Ancak NoSQL SQL'i tamamen değiştirmemiştir; hâlâ ilişkisel model yaygın kullanımdadır.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.